

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(10);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());

        wait(NULL);
    }

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ gcc fork1.c -o fork1
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ ./fork1
Child Process
Parent Process
PID : 587
PID : 588
PPID : 587
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ |
```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;

    for(i=0;i<3;i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID=%d\n",i+1,getpid());
            return 0;
        }
    }

    for(i=0;i<3;i++)
    {
        wait(NULL);
    }

    printf("All children completed.\n");

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ nano MultiChild.c
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ gcc MultiChild.c -o MultiChild
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ ./MultiChild
Child 1 PID=633
Child 2 PID=634
Child 3 PID=635
All children completed.
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ |
```